

# Playwright Test Sharding

## The Complete Field Guide

*Turn a 40 minute suite into a 5 minute one, and explain it in an interview.*

Your suite passes. That was never the problem. The problem is that it takes 38 minutes, so nobody runs it before merging, so it breaks on main, so people stop trusting it. A slow suite quietly becomes an ignored suite.

Sharding is how you fix that without deleting a single test. This guide takes you from the one line definition to a working GitHub Actions pipeline, and ends with the questions interviewers actually ask. Read it once, then keep it open the day you wire it up.

### Remember:

- ✓ Sharding does not make individual tests faster. It makes the wall clock shorter.
- ✓ Every concept here is followed by the exact command or config that implements it.

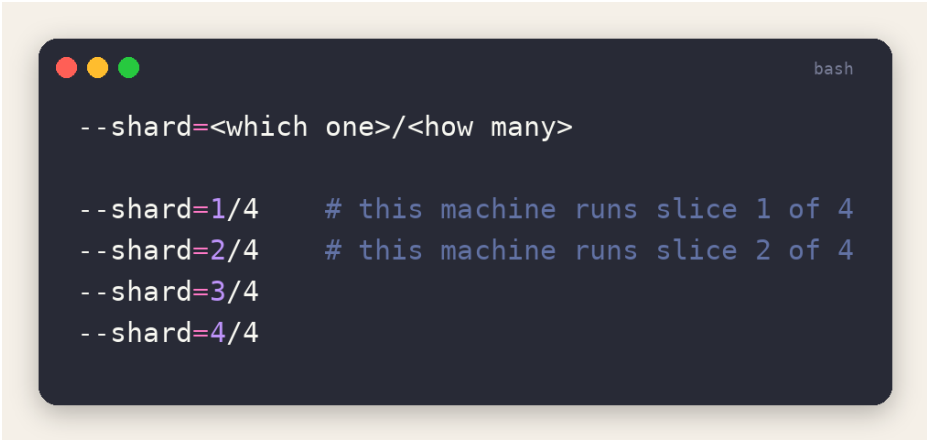
## What is sharding, in one line?

Sharding splits your test suite into slices, runs each slice on a separate machine at the same time, and merges the results back into one report.

Three words carry the whole idea:

- **Split** the suite into equal slices.
- **Distribute** those slices across separate machines or containers.
- **Merge** the per machine reports back into one report you can actually read.

The flag that does it looks like this:

A terminal window with a dark background and light-colored text. The window has three colored window control buttons (red, yellow, green) in the top-left corner and the word 'bash' in the top-right corner. The text inside the terminal shows the syntax for the --shard flag, followed by four specific examples of how to use it to run slices 1 through 4 of a 4-part suite.

```
--shard=<which one>/<how many>

--shard=1/4    # this machine runs slice 1 of 4
--shard=2/4    # this machine runs slice 2 of 4
--shard=3/4
--shard=4/4
```

The first number is which slice this machine runs. The second is how many slices exist in total. Every machine gets the same total, and a different first number. Playwright works out the split for you. You never assign tests by hand.

## How is sharding different from parallel workers?

This is the single most misunderstood point in Playwright, and it is the one interviewers use to separate people who read a blog post from people who have shipped it.

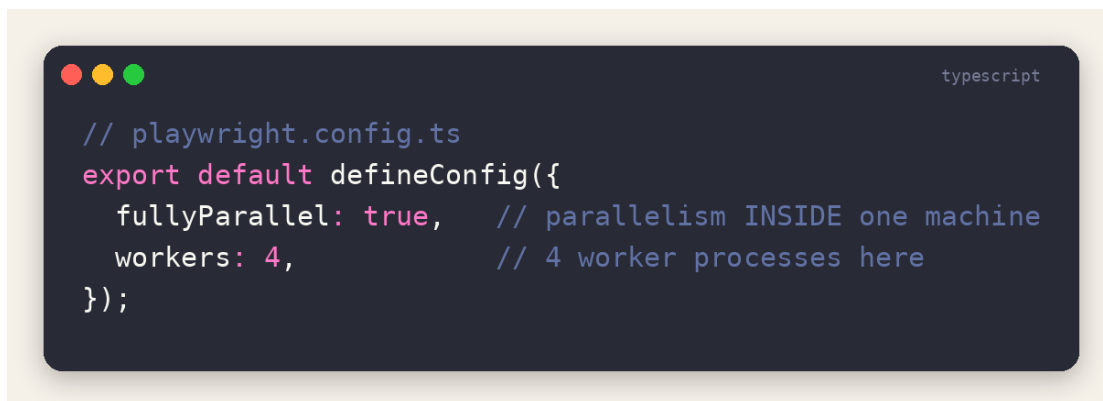
Both give you parallel execution. They are not the same mechanism.

|                  | Parallel Workers                          | Sharding                              |
|------------------|-------------------------------------------|---------------------------------------|
| What it does     | Runs many worker processes on ONE machine | Splits the suite across MANY machines |
| Where you set it | workers in the config                     | --shard=1/4 on the CLI                |
| Ceiling          | CPU cores on that one machine             | How many machines you can spin up     |
| Direction        | Scales UP                                 | Scales OUT                            |
| Reports          | One report already                        | One report per shard, must be merged  |

Here is the mental test that settles it every time:

**Count the machines. One machine running many processes is parallel workers. Many machines each running a slice is sharding.**

Workers are configured inside Playwright:

A screenshot of a code editor window with a dark background and light-colored text. The window has three colored window control buttons (red, yellow, green) in the top-left corner. The text inside is TypeScript code for a Playwright configuration file. The code defines a configuration object with 'fullyParallel' set to true and 'workers' set to 4. Comments explain that 'fullyParallel' is for parallelism inside one machine and 'workers' is for the number of worker processes. The file name 'playwright.config.ts' is visible in the top-left of the editor area, and 'typescript' is in the top-right.

```
// playwright.config.ts
export default defineConfig({
  fullyParallel: true,    // parallelism INSIDE one machine
  workers: 4,            // 4 worker processes here
});
```

Sharding happens outside Playwright, in whatever launches the runs: your CI matrix, Docker Compose, or four terminals.

They are not rivals. They multiply. Four shards, each running two workers, gives you eight tests executing at any moment.

**Remember:**

- ✓ Workers scale UP. Sharding scales OUT.
- ✓ `fullyParallel: true` is what lets tests inside a file run in parallel at all.
- ✓ Using both together is the normal production setup, not an exotic one.

## How do you run your first shard?

Before running anything, look at what a shard would contain. This costs nothing and it is the fastest way to build intuition:



```
bash
# see which tests land in shard 1 (does not run them)
npx playwright test --shard=1/4 --list

# actually run shard 1
npx playwright test --shard=1/4
```

Run that with 1/4, then 2/4, and compare the lists. You will see Playwright hand each shard a different set of tests, with no overlap and nothing missed.

### The detail almost every guide skips

Playwright balances shards by test count, not by duration. If shard 3 happens to receive your three slowest tests, it finishes last and your whole pipeline waits for it. Your total time is set by the slowest shard, never the average.

So if one shard is consistently the straggler, the fix is not more shards. The fix is finding out why those tests are slow.

## Why did you get four reports instead of one?

The first time people shard, this is the moment it stops feeling like a win. Four machines produced four separate HTML reports, each showing a quarter of the tests, and none of them tells you whether the build is green.

The fix is the blob reporter. Blob is not a human readable report. It is an intermediate format designed to be merged.



```
// playwright.config.ts
export default defineConfig({
  fullyParallel: true,
  reporter: [
    ["blob"], // required to merge
    ["html", { open: "never" }],
  ],
});
```

Each shard now writes a blob file. Collect them in one folder and merge:



```
# each shard produced blob-report/report-<n>.zip
# collect them into one folder, then merge

npx playwright merge-reports --reporter=html ./all-blob-reports

npx playwright show-report
```

That produces a single HTML report containing all tests from all shards, as if they had run on one machine.

**Remember:**

- ✓ No blob reporter means no merge. This is the number one sharding failure.
- ✓ Blob is for machines. HTML is for humans. You need both.
- ✓ Merge is a separate step that runs after every shard has finished.

## How do you shard in GitHub Actions?

This is where sharding actually pays off, because CI can give you four fresh machines for free. The matrix strategy creates one job per shard:

```
name: E2E Tests

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false      # one bad shard must not kill others
      matrix:
        shard: [1, 2, 3, 4]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npx playwright install --with-deps

      - name: Run shard ${ matrix.shard }
        run: npx playwright test --shard=${ matrix.shard }/4

      - name: Upload blob report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: blob-report-${ matrix.shard }
          path: blob-report/
          retention-days: 1
```

### Two lines that matter more than they look

**fail-fast: false** is not optional. Without it, the moment shard 2 fails, GitHub cancels shards 1, 3 and 4. You lose the results you were about to merge, and you learn about one failure instead of all of them.

**if: always()** on the upload step means a shard still hands over its report when tests fail. Failed runs are exactly the ones you want to read.

Now the second job, which waits for every shard and merges:

```
merge-reports:
  needs: test
  if: always()           # merge even when a shard failed
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
    - run: npm ci

    - name: Download all blob reports
      uses: actions/download-artifact@v4
      with:
        path: all-blob-reports
        pattern: blob-report-*
        merge-multiple: true    # the step everyone forgets

    - name: Merge into one HTML report
      run: |
        npx playwright merge-reports \
          --reporter=html ./all-blob-reports

    - uses: actions/upload-artifact@v4
      with:
        name: html-report
        path: playwright-report/
```

### The step everyone forgets

**merge-multiple: true** is the one that quietly breaks pipelines. Without it, each artifact lands in its own subfolder, merge-reports finds no blob files at the path you gave it, and it cheerfully produces an empty report showing zero tests. Nothing errors. You just get a report that is wrong.

**Remember:**

- ✓ fail-fast: false, or one failing shard kills the others.
- ✓ if: always() on uploads, or failed shards report nothing.
- ✓ merge-multiple: true, or you merge an empty folder and get a zero test report.
- ✓ The merge job needs if: always() too, or it skips whenever a shard fails.

## How do you shard locally with Docker?

You do not need CI to prove this works. Docker Compose gives you real isolated containers on your own machine, which is the closest local equivalent to separate CI runners:

```
services:
  shard1:
    build: .
    volumes:
      - ./blob-1:/app/blob-report
    command: npx playwright test --shard=1/4

  shard2:
    build: .
    volumes:
      - ./blob-2:/app/blob-report
    command: npx playwright test --shard=2/4
```

Then run everything with one command and merge as usual:

```
docker compose -f docker-compose.shard.yml up --build

# all shards start at once, then merge:
mkdir -p all-blob-reports && cp blob-*/.* all-blob-reports/
npx playwright merge-reports --reporter=html ./all-blob-reports
```

A word of warning from experience: four Chromium containers are heavy. Give Docker at least 4GB of memory, and set workers to 1 inside each container. Otherwise you are asking one laptop to run eight browsers, and containers start dying in ways that look like flaky tests but are not.

## How many shards do you actually need?

More shards is not automatically faster. Every shard pays a fixed startup cost: boot the runner, checkout, npm ci, install browsers. That is often 60 to 90 seconds before a single test runs.

Split 40 tests across 20 shards and you spend more time booting machines than testing. Use this as a starting point, then measure:

| Suite size     | Start with                | Why                              |
|----------------|---------------------------|----------------------------------|
| Under 50 tests | No sharding, workers only | Startup cost outweighs the gain  |
| 50 to 200      | 2 to 4 shards             | Clear win, still cheap           |
| 200 to 500     | 4 to 8 shards             | The sweet spot for most teams    |
| 500 plus       | 8 or more, plus workers   | Combine both to scale out and up |

## When should you not shard?

- Your suite already finishes in under 5 minutes. Fix something else.
- Your tests share state, a database, or a login. Sharding will expose that brutally, and you should fix the isolation first.
- You pay per CI minute and four machines for 90 seconds each costs more than one machine for 4 minutes. Do the math for your own bill.

### Remember:

- ✓ Total time equals your slowest shard, not the average.
- ✓ Every shard pays the setup cost again. Count it before adding more.
- ✓ Sharding amplifies test isolation problems. It does not cause them.

## What breaks, and how do you fix it?

| Symptom                         | Cause and fix                                                                             |
|---------------------------------|-------------------------------------------------------------------------------------------|
| Merged report shows 0 tests     | Blob files were never found. Check <code>merge-multiple: true</code> and the folder path. |
| merge-reports command fails     | Blob reporter is not in the config. Add <code>["blob"]</code> to reporter.                |
| Only one shard's results appear | Artifact names collided. Include the shard number in the artifact name.                   |



|                                     |                                                                                     |
|-------------------------------------|-------------------------------------------------------------------------------------|
| Other shards cancel when one fails  | <code>fail-fast: false</code> is missing from the matrix strategy.                  |
| Failed runs upload nothing          | The upload step needs <code>if: always()</code> .                                   |
| Merge job never runs on failure     | The merge job needs <code>if: always()</code> as well as <code>needs: test</code> . |
| Docker containers die mid run       | Memory. Raise Docker to 4GB and set workers to 1 per container.                     |
| Shards take very different times    | Balancing is by test count, not duration. Investigate your slowest tests.           |
| Tests pass alone, fail when sharded | Shared state between tests. This is a real bug sharding just revealed.              |

One more thing worth adding on day one, so you never commit a report folder:



## What do interviewers actually ask about sharding?

These come up constantly for SDET roles at the package levels people are chasing. Short, correct answers matter more than long ones.

### Q1. What is the difference between sharding and parallel workers?

Workers run multiple processes on one machine and are limited by its CPU cores. Sharding splits the suite across separate machines. Workers scale up, sharding scales out, and you normally use both together.

### Q2. How does Playwright decide which tests go in which shard?

It distributes automatically by test count, not by execution time. You do not assign tests manually. This is also why one shard can finish much later than the others if it happens to hold the slow tests.

### Q3. You sharded across four machines and got four reports. What now?

Add the blob reporter so each shard writes an intermediate blob file, collect all of them into one directory, then run playwright merge-reports to produce a single HTML report.

**Q4. Why is fail-fast: false important in a sharded matrix?**

Without it, the first failing shard cancels the rest. You lose their results and only see one failure instead of the full picture. It also means the merge job has nothing complete to merge.

**Q5. Does sharding make tests run faster?**

No. Each test takes exactly as long as before. Sharding reduces total wall clock time by running slices concurrently. Total time equals the slowest shard.

**Q6. When would you argue against sharding?**

When the suite is small enough that per machine startup cost outweighs the saving, or when tests share state, because sharding will surface that as flakiness rather than fixing it.

**Remember:**

- ✓ Never say sharding makes tests faster. Say it reduces total wall clock time.
- ✓ If you can name the blob reporter and merge-reports unprompted, you sound like someone who has actually run this.

## Where do you go from here?

You now have the mechanism, the config, the CI pipeline, and the interview answers. That puts you ahead of most people who list Playwright on a resume.

Here is the honest gap. Sharding is one piece of a framework. The engineers getting the packages you are aiming for are not hired because they know one flag. They are hired because they can design the whole thing: the architecture, the fixtures, the reporting, the CI, and the reasoning behind each decision.

That is exactly what the SDET Master Class is built around. We construct the Phoenix Test Automation Framework end to end, the same structure that has helped over 8,000 students land roles at companies including Amazon, Goldman Sachs, EPAM and Deloitte. Sharding is one lecture inside it.

If this guide was useful, that course is the same thinking applied to everything else you need.

**Next step**

Build the full framework, not just one flag.

**[courses.techwithjatin.com](https://courses.techwithjatin.com)**

**[courses.techwithjatin.com](https://courses.techwithjatin.com)**

*Tech with Jatin*